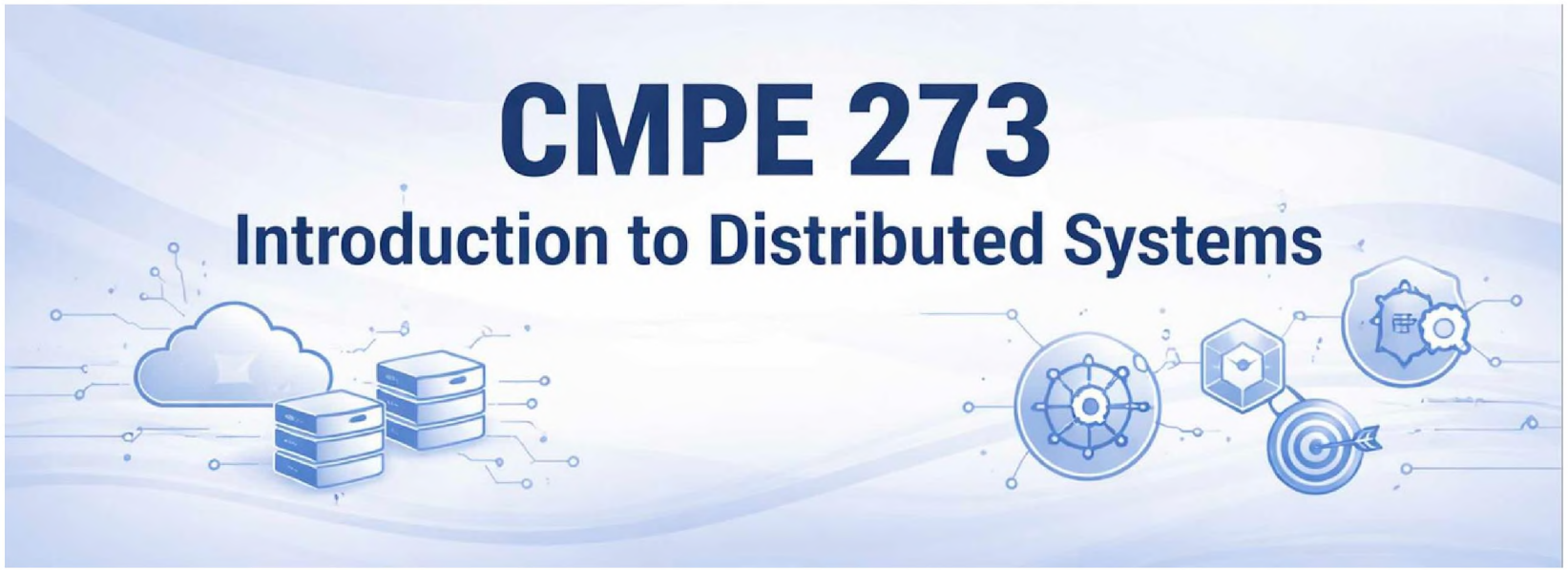


Building systems that work when things go wrong

Week 1: Introduction

Before we start….

Take the ungraded Quiz (Survey)

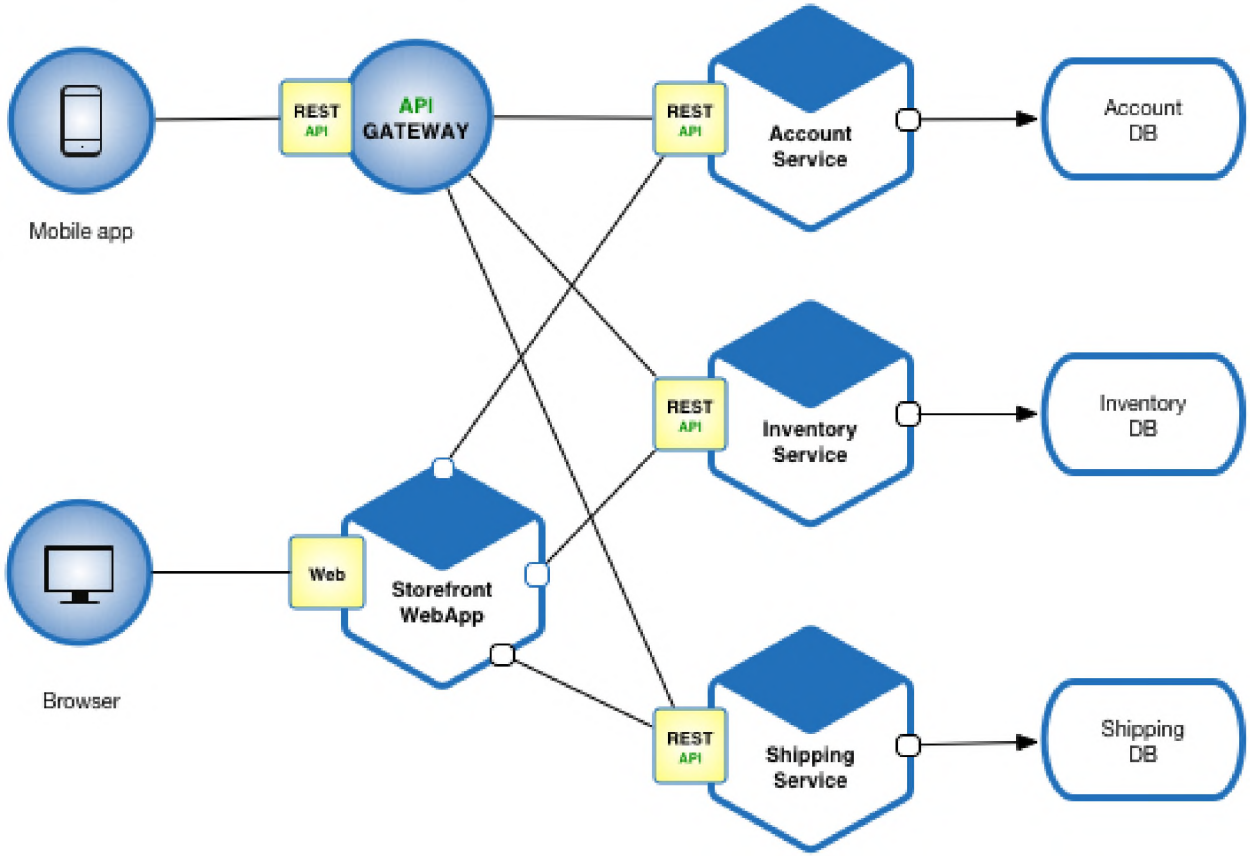
What makes a system Distributed?

A system is distributed when:

- Components run in **different processes**
- Communicate over a **network**
- Can **fail independently**
- Have **no shared clock or memory**

Distributed systems must deal with:

- Latency
- Partial failures
- Concurrency
- Ordering of events
- Consistency vs availability
- Security across trust boundaries
- Observability and diagnosis



Why Distributed Systems Matter

Modern software is distributed by default

- Cloud-native systems
- Global scale
- Fault tolerance
- Team autonomy

Distributed systems power everything you use today

- Cloud platforms
- Enterprise SaaS
- AI systems
- Financial systems
- Global-scale data platforms

But…
Distributed systems fail in ways single-node systems never do.

This course is about understanding, designing, and surviving those failures.

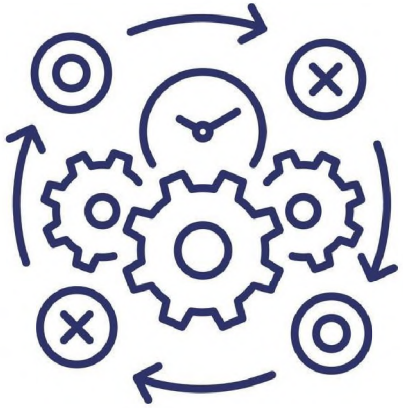
Partial Failure

Single-node system

- If the process crashes, **everything is down**
- Failure is obvious

Example

- A single Java process crashes → the application stops responding



Distributed system

- One service can fail while others keep running
- The system is neither fully “up” nor fully “down”

Example

Service A (auth) is down

Service B (orders) is running

Requests start timing out or failing inconsistently

Network Failure Is Not the Same as Process Failure

Single-node system

- Function calls are in-memory
- If a function returns, it worked
- If it throws, you know immediately
- Time is mostly irrelevant
- Operations are fast and predictable

Distributed system

A request can be:

  - Lost
  - Delayed
  - Duplicated
  - Arrive out of order
- Latency varies
- Slow responses can be worse than crashes

Example

- Service B sends a request to Service A
- Network drops the packet
- Service A actually processed it, but Service B never receives the response

- Service A responds in 10ms normally
- Under load, it responds in 5 seconds
- Service B waits… and waits… and blocks its own resources

You Can’t Assume Order or Consistency

Single-node system

- Operations happen in a well-defined order
- Memory is consistent

Distributed system

- Messages can arrive out of order
- Different nodes see different states

Example

- User updates profile name
- Service A sees the new name
- Service B still sees the old name for a few seconds

There is no global ‘now’ in a distributed system.

Failures Are Often Silent

Single-node system

- Crashes produce stack traces
- Errors are visible

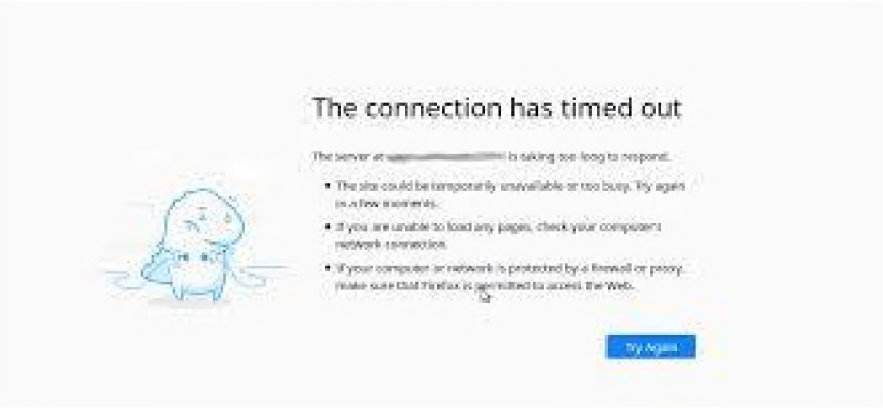
Distributed system

Failures often show up as:

- Timeouts
- Retries
- Degraded performance
- Incomplete data

```
Exception in thread "main" java.lang.IllegalStateException: A book has a null property
        at com.example.myproject.Author.getBookIds(Author.java:38)
        at com.example.myproject.Bootstrap.main(Bootstrap.java:14)
Caused by: java.weblayer.ZzzException
        at com.example.weblayer.Book.getId(WebBook.java:12)
        at com.example.weblayer.Author.getBookIds(WebAuthor.java:38)
Caused by: java.servicelayer.YyyException
        at com.example.servicelayer.Book.getId(BookService.java:220)
        at com.example.servicelayer.Author.getBookIds(AuthorService.java:350)
Caused by: java.componentlayer.NullPointerException
        at com.example.componentlayer.Book.getId(Book.java:22)
        at com.example.componentlayer.Author.getBookIds(Author.java:35)
Caused by: java.lang.daolayer.XxxException
        at com.example.daolayer.Book.getId(BookDao.java:22)
        at com.example.daolayer.Author.getBookIds(AuthorDao.java:35)
        ... 1 more
```

Root Cause may in the middle



Example

A downstream service starts returning 500 errors

Upstream retries hide the failure

Latency slowly increases until users complain

Debugging Requires Reconstruction, Not Inspection

Single-node system

- Attach debugger
- Step through code
- Inspect variables

Distributed system

- No single debugger
- You reconstruct events from:
  - Logs
  - Metrics
  - Traces

Example

- A request touches 6 services
- Fails in one
- You piece together the path after the fact

You debug distributed systems like a crime scene

Why Distributed Systems Fail

The 8 Fallacies:

1. The network is reliable ❌
2. Latency is zero ❌
3. Bandwidth is infinite ❌
4. The network is secure ❌
5. Topology doesn't change ❌
6. There is one administrator ❌
7. Transport cost is zero ❌
8. The network is homogeneous ❌

Failures are often:

- Silent
- Partial
- Intermittent
- Cascading
- Hard to reproduce

Most outages are **not code bugs**.
They are **system design failures**.

What I assume you already know

You already understand:

- APIs and microservices
- Cloud-native deployment
- Containers and orchestration (conceptually)
- Enterprise integration patterns
- Basic DevOps workflows

What you may not have internalized yet:

- Partial failure
- Inconsistent state
- Non-determinism
- Debugging without a “single source of truth”

What you will build this semester

You will design and implement:

- A **multi-service distributed system**
- With real communication and failure modes
- That can be observed, debugged, and explained

This is not a toy project.
This is **portfolio-grade**.

Course Roadmap (14 Weeks)

A High-level flow:

- Architectures
- Processes and runtime models
- Communication
- Coordination
- Naming and discovery
- Consistency and replication
- Fault tolerance
- Observability
- Security
- GenAI-assisted diagnosis
- Hardening and scale
- Demos and failure walkthroughs

Each topic adds **one capability** to your system.

Enterprise-Grade Distributed Systems Stack

Modern distributed systems typically include:

- Stateless services
- Stateful backing stores
- Messaging/event streams
- Service discovery
- Load balancing
- Observability pipelines
- Security layers

Technology you will see:

- Containers and container runtimes
- Message brokers
- Distributed data stores
- Service meshes (conceptually)
- Tracing and logging systems
- Cloud infrastructure primitives

When designing a distributed system, always ask:

1. What happens if this service is slow?
2. What happens if it crashes?
3. What happens if messages arrive twice?
4. What happens if they arrive out of order?
5. How do I know what just happened?

If you can answer these, you're thinking correctly.

Technologies You Will Use

Languages: Python, Go, Java, Rust (your choice)

Core tools:

- Docker
- GitHub
- gRPC / REST
- Kafka / RabbitMQ
- Kubernetes (later)

Open Source Software You'll Likely Use

- HTTP servers/frameworks
- Message brokers (local mode)
- Simple data stores
- Observability tooling

Using Coding Agents (Encouraged)

You are **strongly encouraged** to use:

- Coding assistants
- Code-generation agents
- Documentation summarizers

Rules:

- You must understand what you run
- You must be able to explain design decisions
- Agents are tools, not substitutes for reasoning

Week 1 Lab Overview

Objective: The goal of this lab is to **experience the core characteristics of a distributed system** by building a small but real system locally.

- You will create **two independent services** that communicate over the network, include basic logging, and demonstrate how failures propagate across service boundaries.
- This lab is intentionally simple in scope and foundational in purpose. Everything we build later in the course will build on these ideas.

Service A (Provider)

- Exposes a simple HTTP (or gRPC) API
- Responds to requests from other services
- Runs independently on its own port

Service B (Consumer)

- Exposes its own API
- Calls Service A over the network
- Handles failures gracefully when Service A is unavailable

Even though both services run on the same machine, they must:

- Run as **separate processes**
- Communicate via **network calls**
- Fail independently

What’s Next

Week 2: Architectures

- Microservices, event-driven, service meshes

Come prepared